BERT：深度双向 Transformer 的预训练用于语言理解

Jacob Devlin **Ming-Wei Chang** **Kenton Lee** **Kristina Toutanova**
Google AI Language
{jacobdevlin,mingweichang,kentonl,kristout}@google.com

Abstract

我们提出了一种名为 **BERT** 的新语言表示模型，即来自 Transformer 的**双向编码器表示**（**B**idirectional **E**ncoder **R**epresentations from **T**ransformers）。与最近的语言表示模型 (Peters et al., 2018a; Radford et al., 2018) 不同，BERT 的设计目标是通过在所有层中同时以左右两侧上下文为条件，从无标注文本中预训练深度双向表示。因此，预训练的 BERT 模型只需添加一个额外的输出层进行微调，即可为广泛的 NLP 任务创建最先进的模型，例如问答和自然语言推理，而无需针对特定任务对模型架构进行大规模修改。

BERT 概念简单且实证效果强大。它在十一项 NLP 任务上取得了新的最先进结果，包括将 GLUE 分数提升至 80.5%（绝对提升 7.7 个百分点），MultiNLI 准确率提升至 86.7%（绝对提升 4.6%），SQuAD v1.1 问答测试 F1 提升至 93.2（绝对提升 1.5 分）以及 SQuAD v2.0 测试 F1 提升至 83.1（绝对提升 5.1 分）。

1 引言

语言模型预训练已被证明能有效提升许多 NLP 任务的性能 (Dai and Le, 2015; Peters et al., 2018a; Radford et al., 2018; Howard and Ruder, 2018)。这些任务包括句子级任务，如自然语言推理 (Bowman et al., 2015; Williams et al., 2018) 和释义生成 (Dolan and Brockett, 2005)，其目标是通过整体分析来预测句子间的关系；也包括词元级任务，如命名实体识别和问答，要求模型在词元层面产生细粒度输出 (Tjong Kim Sang and De Meulder, 2003; Rajpurkar et al., 2016)。

将预训练的语言表示应用于下游任务存在两种策略：**基于特征的**方法和**微**□方法。基于特征的方法，如 ELMo (Peters et al., 2018a)，使用包含预训练表示作为附加特征的任务特定架构。微调方法，如生成式预训练 Transformer（OpenAI GPT）(Radford et al., 2018)，引入了最少的任务特定参数，并通过仅微调**所有**预训练参数来在下游任务上进行训练。两种方法在预训练阶段共享相同的目标函数，即使用单向语言模型来学习通用语言表示。

我们认为，当前的技术限制了预训练表示的能力，尤其是对于微调方法。主要局限在于标准语言模型是单向的，这限制了预训练期间可用的架构选择。例如，在 OpenAI GPT 中，作者使用从左到右的架构，每个词元在 Transformer 的自注意力层中只能关注其之前的词元 (Vaswani et al., 2017)。此类限制对于句子级任务是次优的，并且在将基于微调的方法应用于如问答等词元级任务时可能非常不利，因为融合来自两个方向的上下文至关重要。

在本文中，我们通过提出 BERT：来自 Transformer 的**双向编码器表示**来改进基于微调的方法。BERT 通过使用受完形填空任务 (Taylor, 1953) 启发的" 掩码语言模型"（MLM）预训练目标，缓解了前述的单向性限制。掩码语言模型随机掩码输入中的部分词元，目标是仅基于上下文预测被掩码词的原始词汇 ID。与从左到右的语言模型预训练不同，MLM 目标使表示能够融合左右两侧上下文，从而使我们能够预训练深度双向 Transformer。除掩码语言模型外，我们还使用" 下一句预测" 任务来联合预训练文本对表示。本文的贡献如下：

- 我们证明了双向预训练对语言表示的重要性。与 Radford et al. (2018) 使用单向语言模型进行预训练不同，BERT 使用掩码语言模型实现预训练的深度双向表示。这也与 Peters et al. (2018a) 形成对比，后者使用独立训练的从左到右和从右到左语言模型的浅层拼接。
- 我们证明预训练表示减少了对高度工程化的任务特定架构的需求。BERT 是第一个基于微调的表示模型，在大规模句子级**和**词元级任务上都取得了最先进性能，超越了许多任务特定的架构。
- BERT 在十一项 NLP 任务上推进了最先进水平。代码和预训练模型可在 https://github.com/google-research/bert 获取。

2 相关工作

预训练通用语言表示有着悠久的历史，我们在本节中简要回顾最广泛使用的方法。

2.1 无监督的基于特征的方法

学习广泛适用的词语表示已成为活跃的研究领域数十年，包括非神经网络 (Brown et al., 1992; Ando and Zhang, 2005; Blitzer et al., 2006) 和神经网络 (Mikolov et al., 2013; Pennington et al., 2014) 方法。预训练的词嵌入是现代 NLP 系统的重要组成部分，相比从头学习的嵌入有显著提升 (Turian et al., 2010)。为了预训练词嵌入向量，使用了从左到右的语言建模目标 (Mnih and Hinton, 2009)，以及区分左右上下文中正确与错误词的目标 (Mikolov et al., 2013)。

这些方法已被推广到更粗的粒度，如句子嵌入 (Kiros et al., 2015; Logeswaran and Lee, 2018) 或段落嵌入 (Le and Mikolov, 2014)。为了训练句子表示，先前的工作使用了对候选下一句排序 (Jernite et al., 2017; Logeswaran and Lee, 2018)、基于前一语句表示从左到右生成下一句词语 (Kiros et al., 2015) 或去噪自编码器衍生目标 (Hill et al., 2016) 等方法。

ELMo 及其前身 (Peters et al., 2017, 2018a) 在不同维度上推广了传统的词嵌入研究。它们从从左到右和从右到左的语言模型中提取**上下文相关**的特征。每个词元的上下文表示是从左到右和从右到左表示的拼接。在将上下文词嵌入与现有任务特定架构融合时，ELMo 在多个主要的 NLP 基准上推进了最先进水平 (Peters et al., 2018a)，包括问答 (Rajpurkar et al., 2016)、情感分析 (Socher et al., 2013) 和命名实体识别 (Tjong Kim Sang and De Meulder, 2003)。Melamud et al. (2016) 提出通过使用 LSTM 从左右两侧上下文预测单个词来学习上下文表示。与 ELMo 类似，其模型是基于特征的，并非深度双向的。Fedus et al. (2018) 表明完形填空任务可用于提高文本生成模型的鲁棒性。

2.2 无监督的微调方法

与基于特征的方法一样，这一方向的首批工作仅从无标注文本中预训练词嵌入参数 (Collobert and Weston, 2008)。

最近，能够产生上下文词元表示的句子或文档编码器已从无标注文本中预训练，并针对有监督的下游任务进行微调 (Dai and Le, 2015; Howard and Ruder, 2018; Radford et al., 2018)。这些方法的优势在于需要从头学习的参数很少。至少部分归功于此优势，OpenAI GPT (Radford et al., 2018) 在 GLUE 基准 (Wang et al., 2018a) 的许多句子级任务上取得了先前的最先进结果。从左到右的语言建模和自编码器目标已被用于预训练此类模型 (Howard and Ruder, 2018; Radford et al., 2018; Dai and Le, 2015)。

2.3 来自有监督数据的迁移学习

也有研究表明从大数据集的有监督任务中进行有效迁移是可行的，如自然语言推理 (Conneau et al., 2017) 和机器翻译 (McCann et al., 2017)。计算机视觉研究也证明了从大型预训练模型进行迁移学习的重要性，其中一个有效方案是微调用 ImageNet (Deng et al., 2009; Yosinski et al., 2014) 预训练的模型。

3 BERT

我们在本节中介绍 BERT 及其详细实现。我们的框架包含两个步骤：☒☒☒和**微**☒。在预训练期间，模型通过不同的预训练任务在无标注数据上进行训练。对于微调，BERT 模型首先用预训练参数初始化，然后使用来自下游任务的标注数据对所有参数进行微调。每个下游任务都有各自微调后的模型，尽管它们用相同的预训练参数初始化。图 1 中的问答示例将作为本节贯穿始终的说明性例子。

BERT 的一个显著特征是其跨不同任务的统一架构。预训练架构和最终的下游架构之间几乎没有差异。

模型架构 BERT 的模型架构是一个多层双向 Transformer 编码器，基于 Vaswani et al. (2017) 中描述的原始实现并在 `tensor2tensor` 库中发布。[1] 由于 Transformer 的使用已变得普遍，且我们的实现与原始版本几乎相同，我们将省略对模型架构详尽的背景描述，并建议读者参考 Vaswani et al. (2017) 以及诸如"The Annotated Transformer" 等优秀指南。[2]

在本文中，我们用 L 表示层数（即 Transformer 块的数量），用 H 表示隐藏层大小，用 A 表示自注意力头的数量。[3] 我们主要报告两种模型尺寸的结果：$\mathbf{BERT_{BASE}}$（L=12, H=768, A=12, 总参数量=110M）和 $\mathbf{BERT_{LARGE}}$（L=24, H=1024, A=16, 总参数量=340M）。

$\text{BERT}_{\text{BASE}}$ 被选为与 OpenAI GPT 具有相同的模型大小，以便进行比较。然而，关键的是 BERT Transformer 使用双向自注意力，而 GPT Transformer 使用的是受限的自注意力，其中每个词元只能关注其左侧的上下文。[4]

[1] https://github.com/tensorflow/tensor2tensor

[2] http://nlp.seas.harvard.edu/2018/04/03/attention.html

[3] 在所有情况下，我们将前馈/滤波器大小设为 $4H$，即 $H = 768$ 时为 3072，$H = 1024$ 时为 4096。

[4] 我们注意到在文献中，双向 Transformer 通常被称为"Transformer 编码器"，而仅左侧上下文的版本被称

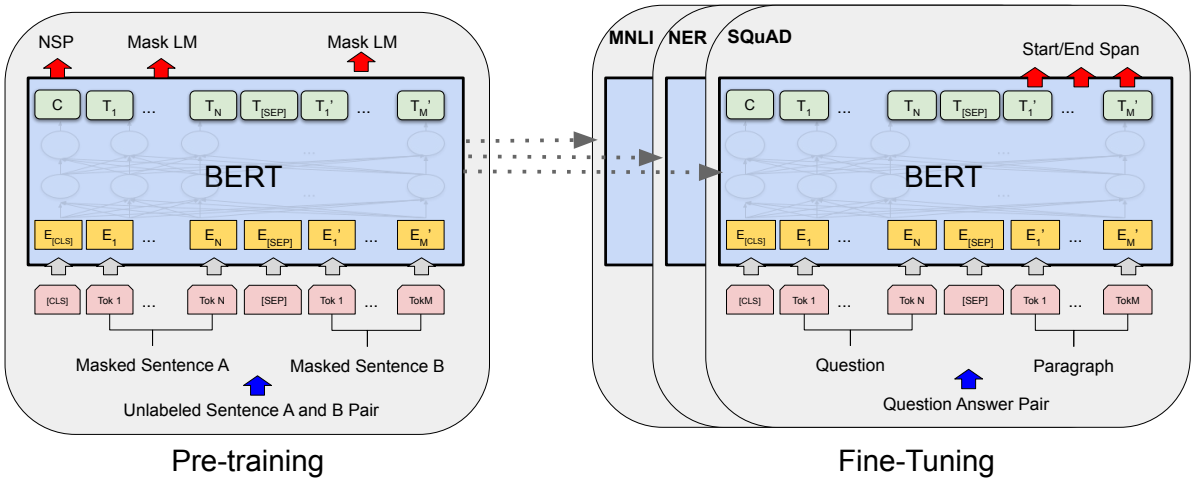


Figure 1: BERT 的整体预训练和微调流程。除输出层外，预训练和微调使用相同的架构。相同的预训练模型参数用于初始化不同下游任务的模型。微调期间，所有参数都参与微调。[CLS] 是添加在每个输入示例之前的特殊符号，[SEP] 是特殊分隔符（例如分隔问题和答案）。

输入/输出表示 为了使 BERT 能够处理多种下游任务，我们的输入表示能够在一个词元序列中明确无误地表示单个句子和句子对（例如 ⟨ 问题，答案 ⟩）。在本文中，”句子”可以是任意连续的文本片段，而不是实际的语法句子。”序列”指的是输入到 BERT 的词元序列，可以是单个句子，也可以是两个句子打包在一起。

我们使用 WordPiece 嵌入 (Wu et al., 2016)，词汇量为 30,000 个词元。每个序列的第一个词元始终是一个特殊的分类词元（[CLS]）。与该词元对应的最终隐藏状态被用作分类任务的聚合序列表示。句子对被一起打包成一个单独的序列。我们通过两种方式区分句子。首先，我们用特殊词元（[SEP]）分隔它们。其次，我们为每个词元添加一个学习到的嵌入，指明它属于句子 A 还是句子 B。如图 1 所示，我们记输入嵌入为 E，特殊 [CLS] 词元的最终隐藏向量为 $C \in \mathbb{R}^H$，第 i^{th} 个输入词元的最终隐藏向量为 $T_i \in \mathbb{R}^H$。

对于给定的词元，其输入表示由对应的词元嵌入、分段嵌入和位置嵌入求和构建而成。该构建的可视化如图 2 所示。

3.1 预训练 BERT

与 Peters et al. (2018a) 和 Radford et al. (2018) 不同，我们不使用传统的从左到右或从右到左的语言模型来预训练 BERT。取而代之，我们使用本节中描述的两个无监督任务来预训练 BERT。这一步在图 1 的左侧部分中展示。

任务 #1：掩码语言模型 直观上，我们有理由相信深度双向模型严格地比从左到右的模型或从左到右与从右到左模型的浅层拼接更强大。不幸的是，标准条件语言模型只能以从左到右或从右到左的方式训练，因为双向条件化将允许每个词间接地”看到自身”，且模型可以在多层上下文中轻松地预测目标词。

为了训练深度双向表示，我们随机掩码一定比例的输入词元，然后预测这些被掩码的词元。我们将此过程称为”掩码 LM”（MLM），尽管在文献中它通常被称为完形填空任务 (Taylor, 1953)。在这种情况下，与掩码词元对应的最终隐藏向量被输入到词汇表上的 softmax 输出层，如同标准 LM 一样。在我们所有的实验中，我们随机掩码每个序列中 15% 的所有 WordPiece 词元。与去噪自编码器 (Vincent et al., 2008) 不同，我们仅预测被掩码的词，而不是重建整个输入。

尽管这使我们能够获得一个双向预训练模型，但缺点是我们在预训练和微调之间制造了不匹配，因为 [MASK] 词元不会在微调期间出现。为了减轻这一问题，我们并不总是将”被掩码”的词替换为实际的 [MASK] 词元。训练数据生成器随机选择 15% 的词元位置用于预测。如果第 i 个词元被选中，我们将该第 i 个词元替换为 (1) 80% 的情况为 [MASK] 词元 (2) 10% 的情况为一个随机词元 (3) 10% 的情况为保持不变的原始第 i 个词元。然后，T_i 将用于通过交叉熵损失来预测原始词元。我们在附录 C.2 中比较了该过程的不同变体。

任务 #2：下一句预测（NSP） 许多重要的下游任务，如问答（QA）和自然语言推理（NLI），都是基于理解两个句子之间的关系，而这是语言建模无法直接捕捉的。为了训练一个理解句子关系的模型，我们预训练一个二值化的下一句预测任务，该任务可以轻松地由任何单语语料库生成。

为”Transformer 解码器”，因为它可用于文本生成。

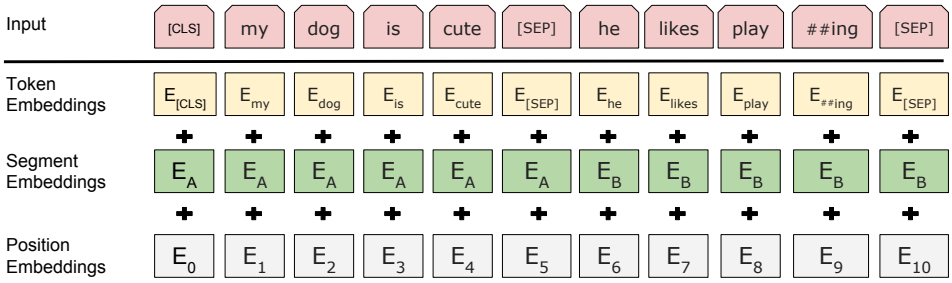


Figure 2: BERT 输入表示。输入嵌入是词元嵌入、分段嵌入和位置嵌入的总和。

具体来说，当为每个预训练示例选择句子 A 和 B 时，50% 的情况下 B 是 A 之后的真实下一句（标记为 IsNext），50% 的情况是来自语料库中的随机句子（标记为 NotNext）。如图 1 所示，C 用于下一句预测（NSP）。[5] 尽管简单，我们在第 5.1 节中证明，预训练此任务对 QA 和 NLI 都非常有益。[6]NSP 任务与 Jernite et al. (2017) 和 Logeswaran and Lee (2018) 中使用的表示学习目标密切相关。然而，在先前的工作中，只有句子嵌入被迁移到下游任务中，而 BERT 将所有参数迁移以初始化端任务模型参数。

预训练数据预训练过程在很大程度上遵循了语言模型预训练的现有文献。对于预训练语料库，我们使用了 BooksCorpus（800M 词语）(Zhu et al., 2015) 和英文维基百科（2,500M 词语）。对于维基百科，我们仅提取文本段落，忽略列表、表格和标题。使用文档级语料库而非经过打乱的句子级语料库（如 Billion Word Benchmark (Chelba et al., 2013)）对于提取长连续序列至关重要。

3.2 微调 BERT

微调十分直接，因为 Transformer 中的自注意力机制使 BERT 能够通过交换适当的输入和输出来建模多种下游任务 —— 无论它们涉及单个文本还是文本对。对于涉及文本对的应用，一种常见模式是先独立编码文本对，然后应用双向交叉注意力，如 Parikh et al. (2016); Seo et al. (2017)。BERT 则使用自注意力机制来统一这两个阶段，因为用自注意力编码拼接的文本对实际上包含了两个句子之间的**双向**交叉注意力。

对于每个任务，我们只需将特定任务的输入和输出接入 BERT，并对所有参数进行端到端微调。在输入端，预训练中的句子 A 和句子 B 类似于 (1) 释义任务中的句子对，(2) 蕴含关系中的假设-前提对，(3) 问答中的问题-段落对，以及 (4) 文本分类或序列标注中的退化文本-$\varnothing$ 对。在输出端，对于词元级任务（如序列标注或问答），词元表示被馈送至输出层；对于分类任务（如蕴含关系或情感分析），[CLS] 表示被馈送至输出层。

与预训练相比，微调相对廉价。本文中的所有结果都可以由完全相同的预训练模型开始，在单个 Cloud TPU 上最多 1 小时内，或在 GPU 上几小时内复现。[7]我们在第 4 节的相应子节中描述了每个任务的特定细节。更多细节可在附录 A.5 中找到。

4 实验

在本节中，我们展示 BERT 在 11 项 NLP 任务上的微调结果。

4.1 GLUE

通用语言理解评估（GLUE）基准 (Wang et al., 2018a) 是多样化的自然语言理解任务集合。GLUE 数据集的详细描述见附录 B.1。

为了在 GLUE 上进行微调，我们按第 3 节所述表示输入序列（单个句子或句子对），并使用与第一个输入词元（[CLS]）对应的最终隐藏向量 $C \in \mathbb{R}^H$ 作为聚合表示。微调期间引入的唯一新参数是分类层权重 $W \in \mathbb{R}^{K \times H}$，其中 K 是标签数量。我们使用 C 和 W 计算标准分类损失，即 $\log(\text{softmax}(CW^T))$。

我们对所有 GLUE 任务使用批量大小 32 并对数据进行 3 个 epoch 的微调。对于每个任务，我们在 Dev 集上选择最佳的微调学习率（在 5e-5、4e-5、3e-5 和 2e-5 中选择）。此外，对于 $\text{BERT}_{\text{LARGE}}$，我们发现微调在小数据集上有时不稳定，因此我们进行了多次随机重启，并选择在 Dev 集上最佳的模型。使用随机重启时，我们使用相同的预训练检查点但进行不同的微调数据混洗和分类器层初始化。[9]

结果如表 1 所示。$\text{BERT}_{\text{BASE}}$ 和 $\text{BERT}_{\text{LARGE}}$ 在所有任务上均以显著优势超越所有系统，相对于

[5]最终模型在 NSP 上达到 97%-98% 的准确率。

[6]向量 C 在没有微调的情况下并不是一个有意义的句子表示，因为它是通过 NSP 训练的。

[7]例如，BERT SQuAD 模型可以在单个 Cloud TPU 上大约 30 分钟内训练完成，达到 Dev F1 得分 91.0%。

[8]See (10) in https://gluebenchmark.com/faq.

[9]GLUE 数据集的分发不包含测试标签，我们只为 $\text{BERT}_{\text{BASE}}$ 和 $\text{BERT}_{\text{LARGE}}$ 分别向 GLUE 评估服务器提交了一次。

System	MNLI-(m/mm) 392k	QQP 363k	QNLI 108k	SST-2 67k	CoLA 8.5k	STS-B 5.7k	MRPC 3.5k	RTE 2.5k	**Average** -
Pre-OpenAI SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.8	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	87.4	91.3	45.4	80.0	82.3	56.0	75.1
$\text{BERT}_{\text{BASE}}$	84.6/83.4	71.2	90.5	93.5	52.1	85.8	88.9	66.4	79.6
$\text{BERT}_{\text{LARGE}}$	**86.7/85.9**	**72.1**	**92.7**	**94.9**	**60.5**	**86.5**	**89.3**	**70.1**	**82.1**

Table 1: GLUE Test results, scored by the evaluation server (https://gluebenchmark.com/leaderboard). The number below each task denotes the number of training examples. The "Average" column is slightly different than the official GLUE score, since we exclude the problematic WNLI set.[8] BERT and OpenAI GPT are single-model, single task. F1 scores are reported for QQP and MRPC, Spearman correlations are reported for STS-B, and accuracy scores are reported for the other tasks. We exclude entries that use BERT as one of their components.

先前的最高水平分别获得了 4.5% 和 7.0% 的平均准确率提升。注意，$\text{BERT}_{\text{BASE}}$ 和 OpenAI GPT 在模型架构上除注意力掩码外几乎相同。对于最大且最广泛被报告的 GLUE 任务 MNLI，BERT 取得了 4.6% 的绝对准确率提升。在 GLUE 官方排行榜[10] 上，$\text{BERT}_{\text{LARGE}}$ 得分为 80.5，而 OpenAI GPT 在撰写本文时的得分为 72.8。

我们发现 $\text{BERT}_{\text{LARGE}}$ 在所有任务上都显著优于 $\text{BERT}_{\text{BASE}}$，尤其是那些训练数据极少的任务。模型规模的影响在第 5.2 节中进行了更深入的探讨。

4.2 SQuAD v1.1

斯坦福问答数据集（SQuAD v1.1）包含 100k 个众包的问题/答案对 (Rajpurkar et al., 2016)。给定一个问题和一个包含答案的维基百科段落，任务是预测段落中答案文本的跨度。

如图 1 所示，在问答任务中，我们将输入的问题和段落表示为单个打包序列，其中问题使用 A 嵌入，段落使用 B 嵌入。微调期间只引入了一个起始向量 $S \in \mathbb{R}^H$ 和一个结束向量 $E \in \mathbb{R}^H$。词 i 成为答案跨度起始位置的概率通过 T_i 和 S 的点积再经过段落中所有词的 softmax 计算：$P_i = \frac{e^{S \cdot T_i}}{\sum_j e^{S \cdot T_j}}$。类似的公式用于答案跨度的结束位置。从位置 i 到位置 j 的候选跨度得分定义为 $S \cdot T_i + E \cdot T_j$，满足 $j \geq i$ 的最高得分跨度被用作预测。训练目标是正确起始和结束位置的对数似然之和。我们使用学习率 5e-5 和批量大小 32 进行 3 个 epoch 的微调。

表 2 展示了顶级的排行榜参赛结果以及顶级已发表系统 (Seo et al., 2017; Clark and Gardner, 2018; Peters et al., 2018a; Hu et al., 2018) 的结果。SQuAD 排行榜的前几名结果没有公开的、最新的系统描述，[11] 且允许在训练时使用任何公开数据。因此，我们在系统中使用了适度的数据增强，即先在 TriviaQA (Joshi et al., 2017) 上进行微调，然后在 SQuAD 上进行微调。

System	Dev EM	Dev F1	Test EM	Test F1
Top Leaderboard Systems (Dec 10th, 2018)				
Human	-	-	82.3	91.2
#1 Ensemble - nlnet	-	-	86.0	91.7
#2 Ensemble - QANet	-	-	84.5	90.5
Published				
BiDAF+ELMo (Single)	-	85.6	-	85.8
R.M. Reader (Ensemble)	81.2	87.9	82.3	88.5
Ours				
$\text{BERT}_{\text{BASE}}$ (Single)	80.8	88.5	-	-
$\text{BERT}_{\text{LARGE}}$ (Single)	84.1	90.9	-	-
$\text{BERT}_{\text{LARGE}}$ (Ensemble)	85.8	91.8	-	-
$\text{BERT}_{\text{LARGE}}$ (Sgl.+TriviaQA)	**84.2**	**91.1**	**85.1**	**91.8**
$\text{BERT}_{\text{LARGE}}$ (Ens.+TriviaQA)	**86.2**	**92.2**	**87.4**	**93.2**

Table 2: SQuAD 1.1 results. The BERT ensemble is 7x systems which use different pre-training checkpoints and fine-tuning seeds.

我们表现最佳的系统在集成模型中比排行榜第一名高出 +1.5 F1，作为单系统高出 +1.3 F1。事实上，我们的单个 BERT 模型在 F1 得分上就超越了顶级的集成系统。如果不使用 TriviaQA 微调数据，我们仅损失 0.1-0.4 F1，仍以较大优势超越所有现有系统。[12]

4.3 SQuAD v2.0

SQuAD 2.0 任务通过允许提供的段落中可能不存在简短答案来扩展 SQuAD 1.1 的问题定义，使得问题更加贴近真实场景。

我们采用一种简单的方法将 SQuAD v1.1 BERT 模型扩展到此任务。我们将没有答案的

[10] https://gluebenchmark.com/leaderboard

[11] QANet 在 Yu et al. (2018) 中有描述，但该系统在发表后已有大幅改进。

[12] 我们使用的 TriviaQA 数据包含来自 TriviaQA-Wiki 的段落，取其前 400 个词元的文档内容，且这些文档至少包含一个提供的可能答案。

System	Dev		Test	
	EM	F1	EM	F1
Top Leaderboard Systems (Dec 10th, 2018)				
Human	86.3	89.0	86.9	89.5
#1 Single - MIR-MRC (F-Net)	-	-	74.8	78.0
#2 Single - nlnet	-	-	74.2	77.1
Published				
unet (Ensemble)	-	-	71.4	74.9
SLQA+ (Single)	-		71.4	74.4
Ours				
$\text{BERT}_{\text{LARGE}}$ (Single)	78.7	81.9	80.0	83.1

Table 3: SQuAD 2.0 results. We exclude entries that use BERT as one of their components.

System	Dev	Test
ESIM+GloVe	51.9	52.7
ESIM+ELMo	59.1	59.2
OpenAI GPT	-	78.0
$\text{BERT}_{\text{BASE}}$	81.6	-
$\text{BERT}_{\text{LARGE}}$	**86.6**	**86.3**
Human (expert)†	-	85.0
Human (5 annotations)†	-	88.0

Table 4: SWAG Dev and Test accuracies. †Human performance is measured with 100 samples, as reported in the SWAG paper.

Tasks	Dev Set				
	MNLI-m (Acc)	QNLI (Acc)	MRPC (Acc)	SST-2 (Acc)	SQuAD (F1)
$\text{BERT}_{\text{BASE}}$	84.4	88.4	86.7	92.7	88.5
No NSP	83.9	84.9	86.5	92.6	87.9
LTR & No NSP	82.1	84.3	77.5	92.1	77.8
+ BiLSTM	82.1	84.1	75.7	91.6	84.9

Table 5: Ablation over the pre-training tasks using the $\text{BERT}_{\text{BASE}}$ architecture. "No NSP" is trained without the next sentence prediction task. "LTR & No NSP" is trained as a left-to-right LM without the next sentence prediction, like OpenAI GPT. "+ BiLSTM" adds a randomly initialized BiLSTM on top of the "LTR + No NSP" model during fine-tuning.

题目视为答案跨度的起始和结束位置均在 [CLS] 词元处。起始和结束答案跨度位置的概率空间被扩展以包含 [CLS] 词元的位置。对于预测，我们比较无答案跨度的得分：$s_{\texttt{null}} = S{\cdot}C + E{\cdot}C$ 与最佳非空跨度得分 $\hat{s_{i,j}} = \max_{j \geq i} S{\cdot}T_i + E{\cdot}T_j$。当 $\hat{s_{i,j}} > s_{\texttt{null}} + \tau$ 时我们预测非空答案，其中阈值 τ 在开发集上选择以最大化 F1。我们未在此模型中使用 TriviaQA 数据。我们使用学习率 5e-5 和批量大小 48 进行 2 个 epoch 的微调。

与先前排行榜参赛结果和顶级已发表工作 (Sun et al., 2018; Wang et al., 2018b) 相比的结果展示在表 3 中，其中排除了使用 BERT 作为组件的系统。我们观察到相比于先前最佳系统有 +5.1 F1 的提升。

4.4 SWAG

具有对抗生成的场景（SWAG）数据集包含 113k 个句子对补全示例，用于评估具有常识基础的推理 (Zellers et al., 2018)。给定一个句子，任务是从四个候选中选择最合理的续写。

在 SWAG 数据集上进行微调时，我们构建四个输入序列，每个包含给定句子（句子 A）与一个候选续写（句子 B）的拼接。唯一引入的特定任务参数是一个向量，其与 [CLS] 词元表示 C 的点积表示每个选项的得分，并通过 softmax 层进行归一化。

我们使用学习率 2e-5 和批量大小 16 对模型进行 3 个 epoch 的微调。结果展示在表 4 中。$\text{BERT}_{\text{LARGE}}$ 比作者基线 ESIM+ELMo 系统高出 +27.1%，比 OpenAI GPT 高出 8.3%。

5 消融研究

在本节中，我们对 BERT 的多个方面进行了消融实验，以更好地理解它们的相对重要性。更多的消融研究可在附录 C 中找到。

5.1 预训练任务的影响

我们通过使用与 $\text{BERT}_{\text{BASE}}$ 完全相同的预训练数据、微调方案和超参数，评估了两个预训练目标，以证明 BERT 的深度双向性的重要性：

无 NSP：一个使用"掩码 LM"（MLM）训练但没有"下一句预测"（NSP）任务的双向模型。

LTR 且无 NSP：一个使用标准从左到右（LTR）LM 而非 MLM 训练的仅左侧上下文模型。左侧限制也在微调时被应用，因为移除它会引入预训练/微调不匹配，从而降低下游性能。此外，该模型在预训练时没有 NSP 任务。这与 OpenAI GPT 直接可比，但我们使用了更大的训练数据集、我们自己的输入表示和微调方案。

我们首先检查 NSP 任务带来的影响。在表 5 中，我们发现移除 NSP 显著损害了 QNLI、MNLI 和 SQuAD 1.1 上的性能。接下来，我们通过比较"无 NSP"和"LTR 且无 NSP"来评估训练双向表示的影响。LTR 模型在所有任务上的表现都逊于 MLM 模型，在 MRPC 和 SQuAD 上的下降尤为显著。

对于 SQuAD，直觉上很明显 LTR 模型在词元预测上会表现不佳，因为词元级别的隐藏状态没有右侧上下文。为了认真尝试强化 LTR 系统，我

们在其上添加了一个随机初始化的 BiLSTM。这确实显著提高了 SQuAD 上的结果，但结果仍远不如预训练的双向模型。BiLSTM 损害了 GLUE 任务上的性能。

我们承认也可以分别训练 LTR 和 RTL 模型，并将每个词元表示为两个模型的拼接，正如 ELMo 所做的那样。但是：(a) 这比单个双向模型昂贵两倍；(b) 这对于 QA 等任务不直观，因为 RTL 模型无法基于问题对答案进行条件化；(c) 这严格弱于深度双向模型，因为后者可以在每一层同时使用左右上下文。

5.2 模型规模的影响

在本节中，我们探讨模型规模对微调任务准确率的影响。我们训练了数个 BERT 模型，它们具有不同的层数、隐藏单元数和注意力头数，其余超参数和训练过程与前述相同。

在选定的 GLUE 任务上的结果展示在表 6 中。在该表中，我们报告了来自 5 次随机重启微调的 Dev 集平均准确率。我们可以看到，更大的模型在所有四个数据集上都带来了严格的准确率提升，即使对于仅有 3,600 个标注训练样本且与预训练任务差异显著的 MRPC 也是如此。同样令人惊讶的是，我们能够在相对于现有文献已经相当庞大的模型之上取得如此显著的提升。例如，Vaswani et al. (2017) 中探索的最大 Transformer 编码器是 (L=6, H=1024, A=16)，具有 100M 参数，而我们在文献中找到的最大 Transformer 是 (L=64, H=512, A=2)，具有 235M 参数 (Al-Rfou et al., 2018)。相比之下，$\text{BERT}_{\text{BASE}}$ 包含 110M 参数，$\text{BERT}_{\text{LARGE}}$ 包含 340M 参数。

长期以来人们已经知道，增加模型规模会在大规模任务（如机器翻译和语言建模）上带来持续改进，这一点通过表 6 中显示的留存训练数据的 LM 困惑度得以证明。然而，我们相信这是第一项令人信服地证明将模型规模扩展到极致也能在极小的规模任务上带来巨大改进的工作，前提是模型已经过充分的预训练。Peters et al. (2018b) 在

Hyperparams				Dev Set Accuracy		
#L	#H	#A	LM (ppl)	MNLI-m	MRPC	SST-2
3	768	12	5.84	77.9	79.8	88.4
6	768	3	5.24	80.6	82.2	90.7
6	768	12	4.68	81.9	84.8	91.3
12	768	12	3.99	84.4	86.7	92.9
12	1024	16	3.54	85.7	86.9	93.3
24	1024	16	3.23	86.6	87.8	93.7

Table 6: Ablation over BERT model size. #L = the number of layers; #H = hidden size; #A = number of attention heads. "LM (ppl)" is the masked LM perplexity of held-out training data.

将预训练的双向 LM 从两层增加到四层时，对下游任务的影响得出了混合的结果；Melamud et al. (2016) 顺便提到将隐藏维度从 200 增加到 600 有帮助，但进一步增加到 1,000 并未带来进一步的改进。这两项先前工作都使用了基于特征的方法 —— 我们假设，当模型直接在下游任务上进行微调且仅使用极少量的随机初始化额外参数时，即使下游任务数据非常少，特定任务模型也能从更大、更具表达力的预训练表示中受益。

5.3 使用 BERT 的基于特征的方法

到目前为止展示的所有 BERT 结果都使用了微调方法，即向预训练模型添加一个简单的分类层，并联合微调所有参数用于下游任务。然而，基于特征的方法 —— 即从预训练模型中提取固定特征 —— 具有某些优势。首先，并非所有任务都能轻松地用 Transformer 编码器架构表示，因此需要添加特定任务的模型架构。其次，预处理计算昂贵的训练数据表示一次，然后在更廉价的模型上基于此表示运行多次实验，具有显著的计算优势。

在本节中，我们通过将 BERT 应用于 CoNLL-2003 命名实体识别（NER）任务 (Tjong Kim Sang and De Meulder, 2003) 来比较这两种方法。在 BERT 的输入中，我们使用保留大小写的 WordPiece 模型，并包含数据提供的最大文档上下文。按照标准实践，我们将其形式化为标注任务，但在输出中不使用 CRF 层。我们使用第一个子词元的表示作为 NER 标签集上词元级分类器的输入。

为了消融微调方法，我们应用基于特征的方法，即从一层或多层中提取激活值，而不对 BERT 的任何参数进行微调。这些上下文嵌入被用作分类层之前一个随机初始化的两层 768 维 BiLSTM 的输入。

结果展示在表 7 中。$\text{BERT}_{\text{LARGE}}$ 与最先进方法表现相当。最佳性能方法将预训练 Transformer 的前四个隐藏层的词元表示进行拼接，仅比微调整个模型低 0.3 F1。这表明 BERT 对于微调和基于特征的方法都是有效的。

6 结论

最近由语言模型的迁移学习带来的实证改进表明，丰富的无监督预训练是许多语言理解系统不可或缺的组成部分。特别是，这些结果使得即便是低资源任务也能受益于深度单向架构。我们的主要贡献是将这些发现进一步推广到深度**双向**架构，使得同一预训练模型能够成功处理广泛的 NLP 任务集合。

System	Dev F1	Test F1
ELMo (Peters et al., 2018a)	95.7	92.2
CVT (Clark et al., 2018)	-	92.6
CSE (Akbik et al., 2018)	-	**93.1**
Fine-tuning approach		
$\text{BERT}_{\text{LARGE}}$	96.6	92.8
$\text{BERT}_{\text{BASE}}$	96.4	92.4
Feature-based approach ($\text{BERT}_{\text{BASE}}$)		
Embeddings	91.0	-
Second-to-Last Hidden	95.6	-
Last Hidden	94.9	-
Weighted Sum Last Four Hidden	95.9	-
Concat Last Four Hidden	96.1	-
Weighted Sum All 12 Layers	95.5	-

Table 7: CoNLL-2003 Named Entity Recognition results. Hyperparameters were selected using the Dev set. The reported Dev and Test scores are averaged over 5 random restarts using those hyperparameters.

References

Alan Akbik, Duncan Blythe, and Roland Vollgraf. 2018. Contextual string embeddings for sequence labeling. In *Proceedings of the 27th International Conference on Computational Linguistics*, pages 1638–1649.

Rami Al-Rfou, Dokook Choe, Noah Constant, Mandy Guo, and Llion Jones. 2018. Character-level language modeling with deeper self-attention. *arXiv preprint arXiv:1808.04444*.

Rie Kubota Ando and Tong Zhang. 2005. A framework for learning predictive structures from multiple tasks and unlabeled data. *Journal of Machine Learning Research*, 6(Nov):1817–1853.

Luisa Bentivogli, Bernardo Magnini, Ido Dagan, Hoa Trang Dang, and Danilo Giampiccolo. 2009. The fifth PASCAL recognizing textual entailment challenge. In *TAC*. NIST.

John Blitzer, Ryan McDonald, and Fernando Pereira. 2006. Domain adaptation with structural correspondence learning. In *Proceedings of the 2006 conference on empirical methods in natural language processing*, pages 120–128. Association for Computational Linguistics.

Samuel R. Bowman, Gabor Angeli, Christopher Potts, and Christopher D. Manning. 2015. A large annotated corpus for learning natural language inference. In *EMNLP*. Association for Computational Linguistics.

Peter F Brown, Peter V Desouza, Robert L Mercer, Vincent J Della Pietra, and Jenifer C Lai. 1992. Class-based n-gram models of natural language. *Computational linguistics*, 18(4):467–479.

Daniel Cer, Mona Diab, Eneko Agirre, Inigo Lopez-Gazpio, and Lucia Specia. 2017. Semeval-2017 task 1: Semantic textual similarity multilingual and crosslingual focused evaluation. In *Proceedings of the 11th International Workshop on Semantic Evaluation (SemEval-2017)*, pages 1–14, Vancouver, Canada. Association for Computational Linguistics.

Ciprian Chelba, Tomas Mikolov, Mike Schuster, Qi Ge, Thorsten Brants, Phillipp Koehn, and Tony Robinson. 2013. One billion word benchmark for measuring progress in statistical language modeling. *arXiv preprint arXiv:1312.3005*.

Z. Chen, H. Zhang, X. Zhang, and L. Zhao. 2018. Quora question pairs.

Christopher Clark and Matt Gardner. 2018. Simple and effective multi-paragraph reading comprehension. In *ACL*.

Kevin Clark, Minh-Thang Luong, Christopher D Manning, and Quoc Le. 2018. Semi-supervised sequence modeling with cross-view training. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 1914–1925.

Ronan Collobert and Jason Weston. 2008. A unified architecture for natural language processing: Deep neural networks with multitask learning. In *Proceedings of the 25th international conference on Machine learning*, pages 160–167. ACM.

Alexis Conneau, Douwe Kiela, Holger Schwenk, Loïc Barrault, and Antoine Bordes. 2017. Supervised learning of universal sentence representations from natural language inference data. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, pages 670–680, Copenhagen, Denmark. Association for Computational Linguistics.

Andrew M Dai and Quoc V Le. 2015. Semi-supervised sequence learning. In *Advances in neural information processing systems*, pages 3079–3087.

J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei. 2009. ImageNet: A Large-Scale Hierarchical Image Database. In *CVPR09*.

William B Dolan and Chris Brockett. 2005. Automatically constructing a corpus of sentential paraphrases. In *Proceedings of the Third International Workshop on Paraphrasing (IWP2005)*.

William Fedus, Ian Goodfellow, and Andrew M Dai. 2018. Maskgan: Better text generation via filling in the_. *arXiv preprint arXiv:1801.07736*.

Dan Hendrycks and Kevin Gimpel. 2016. Bridging nonlinearities and stochastic regularizers with gaussian error linear units. *CoRR*, abs/1606.08415.

Felix Hill, Kyunghyun Cho, and Anna Korhonen. 2016. Learning distributed representations of sentences from unlabelled data. In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics.

Jeremy Howard and Sebastian Ruder. 2018. Universal language model fine-tuning for text classification. In *ACL*. Association for Computational Linguistics.

Minghao Hu, Yuxing Peng, Zhen Huang, Xipeng Qiu, Furu Wei, and Ming Zhou. 2018. Reinforced mnemonic reader for machine reading comprehension. In *IJCAI*.

Yacine Jernite, Samuel R. Bowman, and David Sontag. 2017. Discourse-based objectives for fast unsupervised sentence representation learning. *CoRR*, abs/1705.00557.

Mandar Joshi, Eunsol Choi, Daniel S Weld, and Luke Zettlemoyer. 2017. Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension. In *ACL*.

Ryan Kiros, Yukun Zhu, Ruslan R Salakhutdinov, Richard Zemel, Raquel Urtasun, Antonio Torralba, and Sanja Fidler. 2015. Skip-thought vectors. In *Advances in neural information processing systems*, pages 3294–3302.

Quoc Le and Tomas Mikolov. 2014. Distributed representations of sentences and documents. In *International Conference on Machine Learning*, pages 1188–1196.

Hector J Levesque, Ernest Davis, and Leora Morgenstern. 2011. The winograd schema challenge. In *Aaai spring symposium: Logical formalizations of commonsense reasoning*, volume 46, page 47.

Lajanugen Logeswaran and Honglak Lee. 2018. An efficient framework for learning sentence representations. In *International Conference on Learning Representations*.

Bryan McCann, James Bradbury, Caiming Xiong, and Richard Socher. 2017. Learned in translation: Contextualized word vectors. In *NIPS*.

Oren Melamud, Jacob Goldberger, and Ido Dagan. 2016. context2vec: Learning generic context embedding with bidirectional LSTM. In *CoNLL*.

Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S Corrado, and Jeff Dean. 2013. Distributed representations of words and phrases and their compositionality. In *Advances in Neural Information Processing Systems 26*, pages 3111–3119. Curran Associates, Inc.

Andriy Mnih and Geoffrey E Hinton. 2009. A scalable hierarchical distributed language model. In D. Koller, D. Schuurmans, Y. Bengio, and L. Bottou, editors, *Advances in Neural Information Processing Systems 21*, pages 1081–1088. Curran Associates, Inc.

Ankur P Parikh, Oscar Täckström, Dipanjan Das, and Jakob Uszkoreit. 2016. A decomposable attention model for natural language inference. In *EMNLP*.

Jeffrey Pennington, Richard Socher, and Christopher D. Manning. 2014. Glove: Global vectors for word representation. In *Empirical Methods in Natural Language Processing (EMNLP)*, pages 1532–1543.

Matthew Peters, Waleed Ammar, Chandra Bhagavatula, and Russell Power. 2017. Semi-supervised sequence tagging with bidirectional language models. In *ACL*.

Matthew Peters, Mark Neumann, Mohit Iyyer, Matt Gardner, Christopher Clark, Kenton Lee, and Luke Zettlemoyer. 2018a. Deep contextualized word representations. In *NAACL*.

Matthew Peters, Mark Neumann, Luke Zettlemoyer, and Wen-tau Yih. 2018b. Dissecting contextual word embeddings: Architecture and representation. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 1499–1509.

Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. 2018. Improving language understanding with unsupervised learning. Technical report, OpenAI.

Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. Squad: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pages 2383–2392.

Minjoon Seo, Aniruddha Kembhavi, Ali Farhadi, and Hannaneh Hajishirzi. 2017. Bidirectional attention flow for machine comprehension. In *ICLR*.

Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D Manning, Andrew Ng, and Christopher Potts. 2013. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of the 2013 conference on empirical methods in natural language processing*, pages 1631–1642.

Fu Sun, Linyang Li, Xipeng Qiu, and Yang Liu. 2018. U-net: Machine reading comprehension with unanswerable questions. *arXiv preprint arXiv:1810.06638*.

Wilson L Taylor. 1953. "Cloze procedure": A new tool for measuring readability. *Journalism Bulletin*, 30(4):415–433.

Erik F Tjong Kim Sang and Fien De Meulder. 2003. Introduction to the conll-2003 shared task: Language-independent named entity recognition. In *CoNLL*.

Joseph Turian, Lev Ratinov, and Yoshua Bengio. 2010. Word representations: A simple and general method for semi-supervised learning. In *Proceedings of the 48th Annual Meeting of the Association for Computational Linguistics*, ACL '10, pages 384–394.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems*, pages 6000–6010.

Pascal Vincent, Hugo Larochelle, Yoshua Bengio, and Pierre-Antoine Manzagol. 2008. Extracting and composing robust features with denoising autoencoders. In *Proceedings of the 25th international conference on Machine learning*, pages 1096–1103. ACM.

Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel Bowman. 2018a. Glue: A multi-task benchmark and analysis platform for natural language understanding. In *Proceedings of the 2018 EMNLP Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*, pages 353–355.

Wei Wang, Ming Yan, and Chen Wu. 2018b. Multi-granularity hierarchical attention fusion networks for reading comprehension and question answering. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics.

Alex Warstadt, Amanpreet Singh, and Samuel R Bowman. 2018. Neural network acceptability judgments. *arXiv preprint arXiv:1805.12471*.

Adina Williams, Nikita Nangia, and Samuel R Bowman. 2018. A broad-coverage challenge corpus for sentence understanding through inference. In *NAACL*.

Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, et al. 2016. Google's neural machine translation system: Bridging the gap between human and machine translation. *arXiv preprint arXiv:1609.08144*.

Jason Yosinski, Jeff Clune, Yoshua Bengio, and Hod Lipson. 2014. How transferable are features in deep neural networks? In *Advances in neural information processing systems*, pages 3320–3328.

Adams Wei Yu, David Dohan, Minh-Thang Luong, Rui Zhao, Kai Chen, Mohammad Norouzi, and Quoc V Le. 2018. QANet: Combining local convolution with global self-attention for reading comprehension. In *ICLR*.

Rowan Zellers, Yonatan Bisk, Roy Schwartz, and Yejin Choi. 2018. Swag: A large-scale adversarial dataset for grounded commonsense inference. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Yukun Zhu, Ryan Kiros, Rich Zemel, Ruslan Salakhutdinov, Raquel Urtasun, Antonio Torralba, and Sanja Fidler. 2015. Aligning books and movies: Towards story-like visual explanations by watching movies and reading books. In *Proceedings of the IEEE international conference on computer vision*, pages 19–27.

Appendix for "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding"

We organize the appendix into three sections:

- Additional implementation details for BERT are presented in Appendix A;
- Additional details for our experiments are presented in Appendix B; and
- Additional ablation studies are presented in Appendix C.

  We present additional ablation studies for BERT including:

  - Effect of Number of Training Steps; and
  - Ablation for Different Masking Procedures.

A Additional Details for BERT

A.1 Illustration of the Pre-training Tasks

We provide examples of the pre-training tasks in the following.

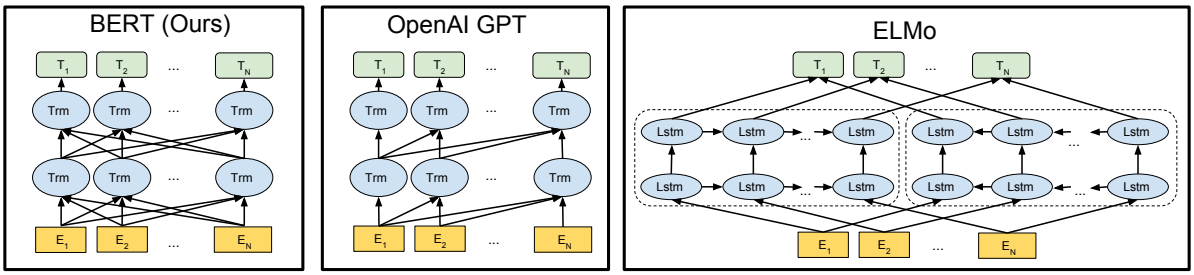


Figure 3: Differences in pre-training model architectures. BERT uses a bidirectional Transformer. OpenAI GPT uses a left-to-right Transformer. ELMo uses the concatenation of independently trained left-to-right and right-to-left LSTMs to generate features for downstream tasks. Among the three, only BERT representations are jointly conditioned on both left and right context in all layers. In addition to the architecture differences, BERT and OpenAI GPT are fine-tuning approaches, while ELMo is a feature-based approach.

Masked LM and the Masking Procedure Assuming the unlabeled sentence is `my dog is hairy`, and during the random masking procedure we chose the 4-th token (which corresponding to `hairy`), our masking procedure can be further illustrated by

- 80% of the time: Replace the word with the `[MASK]` token, e.g., `my dog is hairy` → `my dog is [MASK]`
- 10% of the time: Replace the word with a random word, e.g., `my dog is hairy` → `my dog is apple`
- 10% of the time: Keep the word unchanged, e.g., `my dog is hairy` → `my dog is hairy`. The purpose of this is to bias the representation towards the actual observed word.

The advantage of this procedure is that the Transformer encoder does not know which words it will be asked to predict or which have been replaced by random words, so it is forced to keep a distributional contextual representation of *every* input token. Additionally, because random replacement only occurs for 1.5% of all tokens (i.e., 10% of 15%), this does not seem to harm the model's language understanding capability. In Section C.2, we evaluate the impact this procedure.

Compared to standard langauge model training, the masked LM only make predictions on 15% of tokens in each batch, which suggests that more pre-training steps may be required for the model to converge. In Section C.1 we demonstrate that MLM does converge marginally slower than a left-to-right model (which predicts every token), but the empirical improvements of the MLM model far outweigh the increased training cost.

Next Sentence Prediction The next sentence prediction task can be illustrated in the following examples.

```
Input = [CLS] the man went to [MASK] store [SEP]
        he bought a gallon [MASK] milk [SEP]
Label = IsNext

Input = [CLS] the man [MASK] to the store [SEP]
        penguin [MASK] are flight ##less birds [SEP]
Label = NotNext
```

A.2 Pre-training Procedure

To generate each training input sequence, we sample two spans of text from the corpus, which we refer to as "sentences" even though they are typically much longer than single sentences (but can be shorter also). The first sentence receives the `A` embedding and the second receives the `B` embedding. 50% of the time `B` is the actual next sentence that follows `A` and 50% of the time it is a random sentence, which is done for the "next sentence prediction" task. They are sampled such that the combined length is $\leq$ 512 tokens. The LM masking is applied after WordPiece tokenization with a uniform masking rate of 15%, and no special consideration given to partial word pieces.

We train with batch size of 256 sequences (256 sequences * 512 tokens = 128,000 to-

kens/batch) for 1,000,000 steps, which is approximately 40 epochs over the 3.3 billion word corpus. We use Adam with learning rate of 1e-4, $\beta_1 = 0.9$, $\beta_2 = 0.999$, L2 weight decay of 0.01, learning rate warmup over the first 10,000 steps, and linear decay of the learning rate. We use a dropout probability of 0.1 on all layers. We use a `gelu` activation (Hendrycks and Gimpel, 2016) rather than the standard `relu`, following OpenAI GPT. The training loss is the sum of the mean masked LM likelihood and the mean next sentence prediction likelihood.

Training of $\text{BERT}_{\text{BASE}}$ was performed on 4 Cloud TPUs in Pod configuration (16 TPU chips total).[13] Training of $\text{BERT}_{\text{LARGE}}$ was performed on 16 Cloud TPUs (64 TPU chips total). Each pre-training took 4 days to complete.

Longer sequences are disproportionately expensive because attention is quadratic to the sequence length. To speed up pretraing in our experiments, we pre-train the model with sequence length of 128 for 90% of the steps. Then, we train the rest 10% of the steps of sequence of 512 to learn the positional embeddings.

A.3 Fine-tuning Procedure

For fine-tuning, most model hyperparameters are the same as in pre-training, with the exception of the batch size, learning rate, and number of training epochs. The dropout probability was always kept at 0.1. The optimal hyperparameter values are task-specific, but we found the following range of possible values to work well across all tasks:

- **Batch size**: 16, 32
- **Learning rate (Adam)**: 5e-5, 3e-5, 2e-5
- **Number of epochs**: 2, 3, 4

We also observed that large data sets (e.g., 100k+ labeled training examples) were far less sensitive to hyperparameter choice than small data sets. Fine-tuning is typically very fast, so it is reasonable to simply run an exhaustive search over the above parameters and choose the model that performs best on the development set.

[13] https://cloudplatform.google-blog.com/2018/06/Cloud-TPU-now-offers-preemptible-pricing-and-global-availability.html

A.4 Comparison of BERT, ELMo ,and OpenAI GPT

Here we studies the differences in recent popular representation learning models including ELMo, OpenAI GPT and BERT. The comparisons between the model architectures are shown visually in Figure 3. Note that in addition to the architecture differences, BERT and OpenAI GPT are fine-tuning approaches, while ELMo is a feature-based approach.

The most comparable existing pre-training method to BERT is OpenAI GPT, which trains a left-to-right Transformer LM on a large text corpus. In fact, many of the design decisions in BERT were intentionally made to make it as close to GPT as possible so that the two methods could be minimally compared. The core argument of this work is that the bidirectionality and the two pre-training tasks presented in Section 3.1 account for the majority of the empirical improvements, but we do note that there are several other differences between how BERT and GPT were trained:

- GPT is trained on the BooksCorpus (800M words); BERT is trained on the BooksCorpus (800M words) and Wikipedia (2,500M words).
- GPT uses a sentence separator (`[SEP]`) and classifier token (`[CLS]`) which are only introduced at fine-tuning time; BERT learns `[SEP]`, `[CLS]` and sentence `A/B` embeddings during pre-training.
- GPT was trained for 1M steps with a batch size of 32,000 words; BERT was trained for 1M steps with a batch size of 128,000 words.
- GPT used the same learning rate of 5e-5 for all fine-tuning experiments; BERT chooses a task-specific fine-tuning learning rate which performs the best on the development set.

To isolate the effect of these differences, we perform ablation experiments in Section 5.1 which demonstrate that the majority of the improvements are in fact coming from the two pre-training tasks and the bidirectionality they enable.

A.5 Illustrations of Fine-tuning on Different Tasks

The illustration of fine-tuning BERT on different tasks can be seen in Figure 4. Our task-specific models are formed by incorporating BERT with one additional output layer, so a minimal number of parameters need to be learned from scratch. Among the tasks, (a) and (b) are sequence-level tasks while (c) and (d) are token-level tasks. In the figure, E represents the input embedding, T_i represents the contextual representation of token i, [CLS] is the special symbol for classification output, and [SEP] is the special symbol to separate non-consecutive token sequences.

B Detailed Experimental Setup

B.1 Detailed Descriptions for the GLUE Benchmark Experiments.

Our GLUE results in Table1 are obtained from https://gluebenchmark.com/leaderboard and https://blog.openai.com/language-unsupervised. The GLUE benchmark includes the following datasets, the descriptions of which were originally summarized in Wang et al. (2018a):

MNLI Multi-Genre Natural Language Inference is a large-scale, crowdsourced entailment classification task (Williams et al., 2018). Given a pair of sentences, the goal is to predict whether the second sentence is an *entailment*, *contradiction*, or *neutral* with respect to the first one.

QQP Quora Question Pairs is a binary classification task where the goal is to determine if two questions asked on Quora are semantically equivalent (Chen et al., 2018).

QNLI Question Natural Language Inference is a version of the Stanford Question Answering Dataset (Rajpurkar et al., 2016) which has been converted to a binary classification task (Wang et al., 2018a). The positive examples are (question, sentence) pairs which do contain the correct answer, and the negative examples are (question, sentence) from the same paragraph which do not contain the answer.

SST-2 The Stanford Sentiment Treebank is a binary single-sentence classification task consisting of sentences extracted from movie reviews with human annotations of their sentiment (Socher et al., 2013).

CoLA The Corpus of Linguistic Acceptability is a binary single-sentence classification task, where the goal is to predict whether an English sentence is linguistically "acceptable" or not (Warstadt et al., 2018).

STS-B The Semantic Textual Similarity Benchmark is a collection of sentence pairs drawn from news headlines and other sources (Cer et al., 2017). They were annotated with a score from 1 to 5 denoting how similar the two sentences are in terms of semantic meaning.

MRPC Microsoft Research Paraphrase Corpus consists of sentence pairs automatically extracted from online news sources, with human annotations for whether the sentences in the pair are semantically equivalent (Dolan and Brockett, 2005).

RTE Recognizing Textual Entailment is a binary entailment task similar to MNLI, but with much less training data (Bentivogli et al., 2009).[14]

WNLI Winograd NLI is a small natural language inference dataset (Levesque et al., 2011). The GLUE webpage notes that there are issues with the construction of this dataset, [15] and every trained system that's been submitted to GLUE has performed worse than the 65.1 baseline accuracy of predicting the majority class. We therefore exclude this set to be fair to OpenAI GPT. For our GLUE submission, we always predicted the majority class.

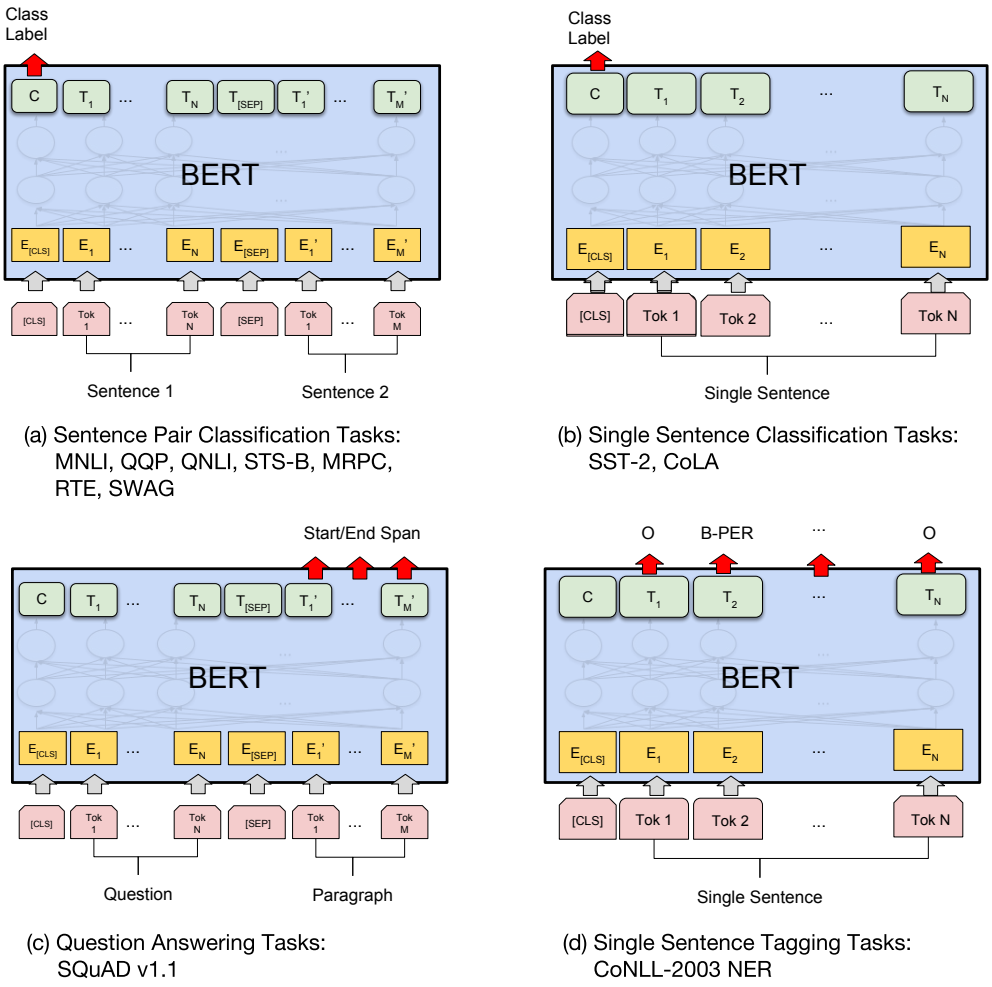
C Additional Ablation Studies

C.1 Effect of Number of Training Steps

Figure 5 presents MNLI Dev accuracy after fine-tuning from a checkpoint that has been pre-trained for k steps. This allows us to answer the following questions:

[14] Note that we only report single-task fine-tuning results in this paper. A multitask fine-tuning approach could potentially push the performance even further. For example, we did observe substantial improvements on RTE from multi-task training with MNLI.

[15] https://gluebenchmark.com/faq



(a) Sentence Pair Classification Tasks: MNLI, QQP, QNLI, STS-B, MRPC, RTE, SWAG

(b) Single Sentence Classification Tasks: SST-2, CoLA

(c) Question Answering Tasks: SQuAD v1.1

(d) Single Sentence Tagging Tasks: CoNLL-2003 NER

Figure 4: Illustrations of Fine-tuning BERT on Different Tasks.

1. Question: Does BERT really need such a large amount of pre-training (128,000 words/batch * 1,000,000 steps) to achieve high fine-tuning accuracy?
   Answer: Yes, $\text{BERT}_{\text{BASE}}$ achieves almost 1.0% additional accuracy on MNLI when trained on 1M steps compared to 500k steps.

2. Question: Does MLM pre-training converge slower than LTR pre-training, since only 15% of words are predicted in each batch rather than every word?
   Answer: The MLM model does converge slightly slower than the LTR model. However, in terms of absolute accuracy the MLM model begins to outperform the LTR model almost immediately.

C.2 Ablation for Different Masking Procedures

In Section 3.1, we mention that BERT uses a mixed strategy for masking the target tokens when pre-training with the masked language model (MLM) objective. The following is an ablation study to evaluate the effect of different masking strategies.

Note that the purpose of the masking strategies is to reduce the mismatch between pre-training and fine-tuning, as the `[MASK]` symbol never appears during the fine-tuning stage. We report the Dev results for both MNLI and NER. For NER, we report both fine-tuning and feature-based approaches, as we expect the mismatch will be amplified for the feature-based approach as the model will not have the chance to adjust the representations.

The results are presented in Table 8. In the table, MASK means that we replace the target

Masking Rates			Dev Set Results		
MASK	SAME	RND	MNLI	NER	
			Fine-tune	Fine-tune	Feature-based
80%	10%	10%	84.2	95.4	94.9
100%	0%	0%	84.3	94.9	94.0
80%	0%	20%	84.1	95.2	94.6
80%	20%	0%	84.4	95.2	94.7
0%	20%	80%	83.7	94.8	94.6
0%	0%	100%	83.6	94.9	94.6

Table 8: Ablation over different masking strategies.

token with the [MASK] symbol for MLM; SAME means that we keep the target token as is; RND means that we replace the target token with another random token.

The numbers in the left part of the table represent the probabilities of the specific strategies used during MLM pre-training (BERT uses 80%, 10%, 10%). The right part of the paper represents the Dev set results. For the feature-based approach, we concatenate the last 4 layers of BERT as the features, which was shown to be the best approach in Section 5.3.

From the table it can be seen that fine-tuning is surprisingly robust to different masking strategies. However, as expected, using only the MASK strategy was problematic when applying the feature-based approach to NER. Interestingly, using only the RND strategy performs much worse than our strategy as well.

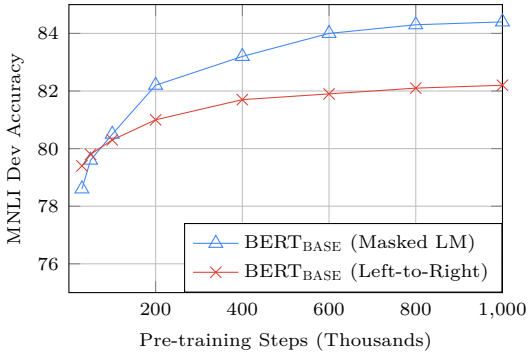


Figure 5: Ablation over number of training steps. This shows the MNLI accuracy after fine-tuning, starting from model parameters that have been pre-trained for k steps. The x-axis is the value of k.